

COMP 272 / COMP 400C · DATA STRUCTURES II

Module 3: Trees I

Hierarchies, Binary Trees and BSTs

Original course slides prepared by Prof. Silva, adapting Data Structures & Algorithms in Java (Goodrich, Goldwasser, Tamassia) and Data Structures and Algorithm Analysis in Java (Weiss). Revised and expanded by Dr. Karima Ennaoui.



What We'll Cover

- | | | |
|----|---------------------|--|
| 01 | Tree Fundamentals | Terminology, the linked representation, the Tree ADT, and traversals |
| 02 | Binary Trees | The special case of ≤ 2 children — properties and traversal by position |
| 03 | Binary Search Trees | Ordering the tree so search, insert, and delete cost $O(h)$ |

Throughout: every structure is examined for what it costs in the worst case, and why that motivates the next one.

01

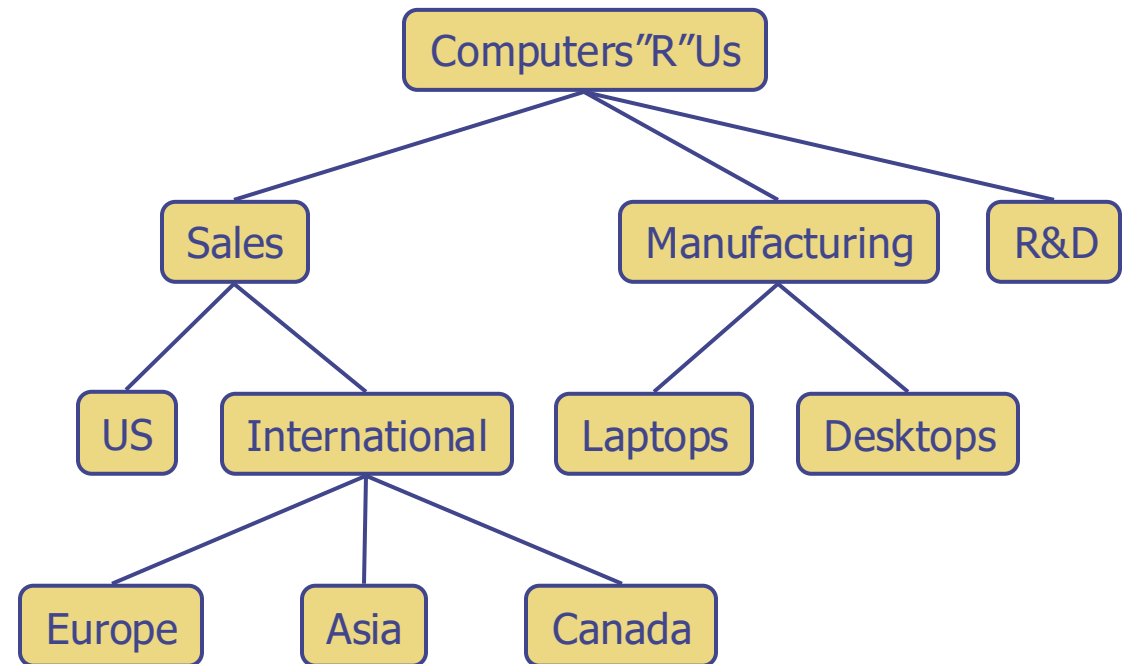
Tree Fundamentals

Hierarchies, vocabulary, the linked representation, the Tree ADT, and traversal order

What Is a Tree?

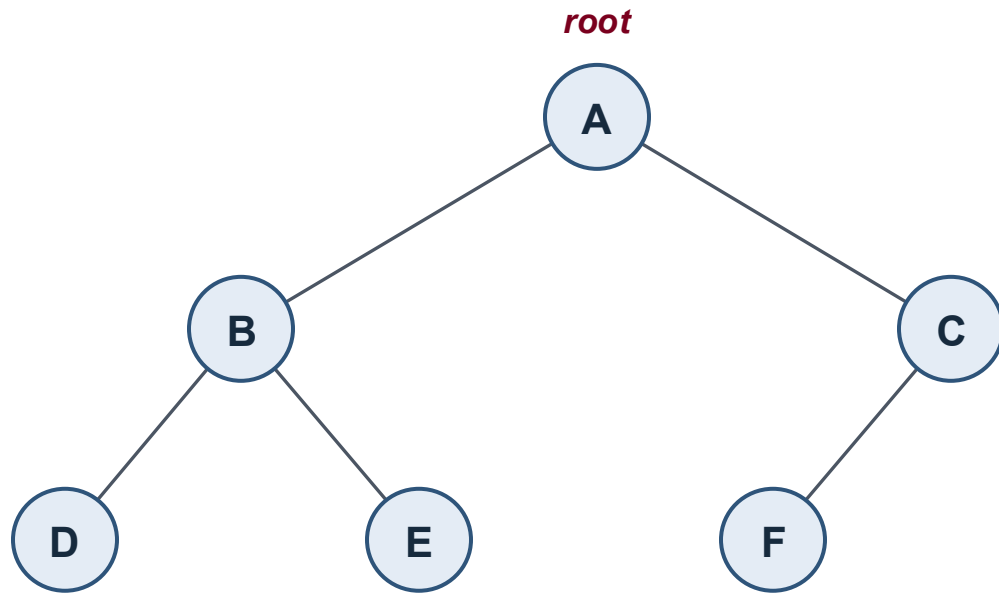
- A tree is an abstract model of a hierarchical structure: nodes connected by a parent–child relationship.
- Exactly one node — the root — has no parent. Every other node has exactly one parent.
- Trees show up constantly in software, not just in data-structures class:
 - File systems (folders contain files and folders)
 - The DOM of every web page you've ever loaded
 - A compiler's parse tree / abstract syntax tree for your Java code
 - Decision trees in machine learning
 - Family trees, org charts, taxonomies, comment threads, JSON/XML documents

Company Organization Tree



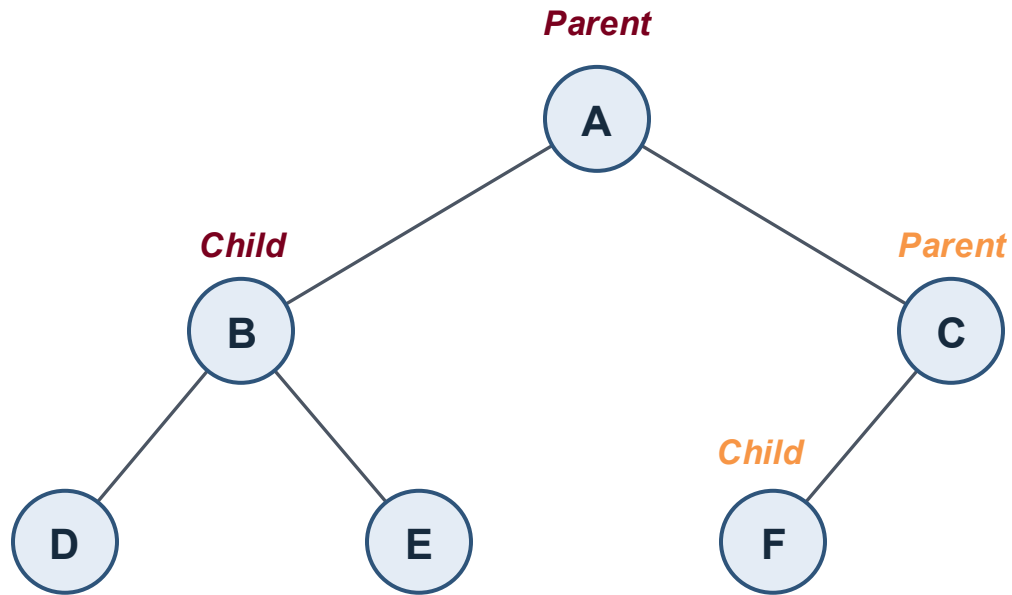


Tree Terminology — Structure



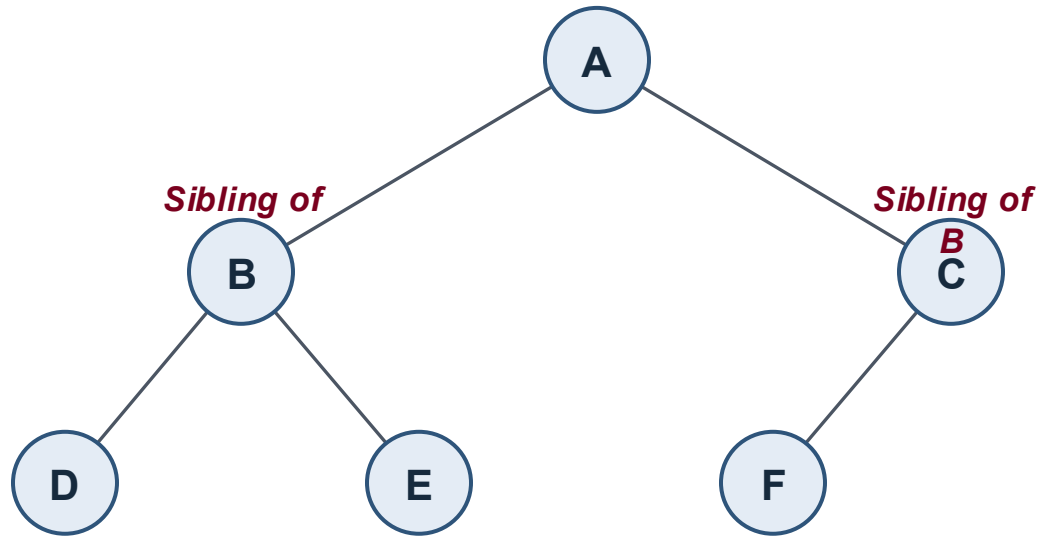
- Root — the node with no parent (A)

Tree Terminology — Structure



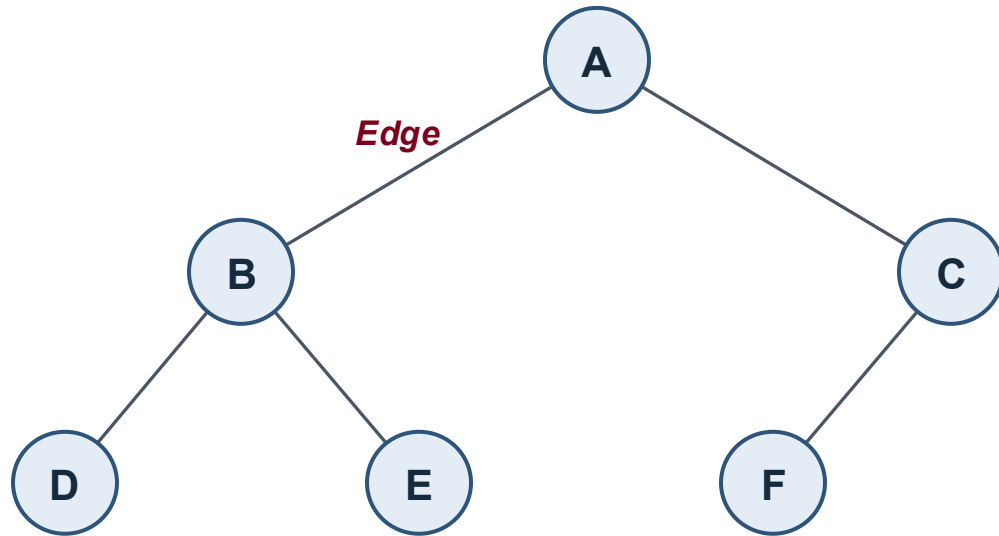
- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B

Tree Terminology — Structure



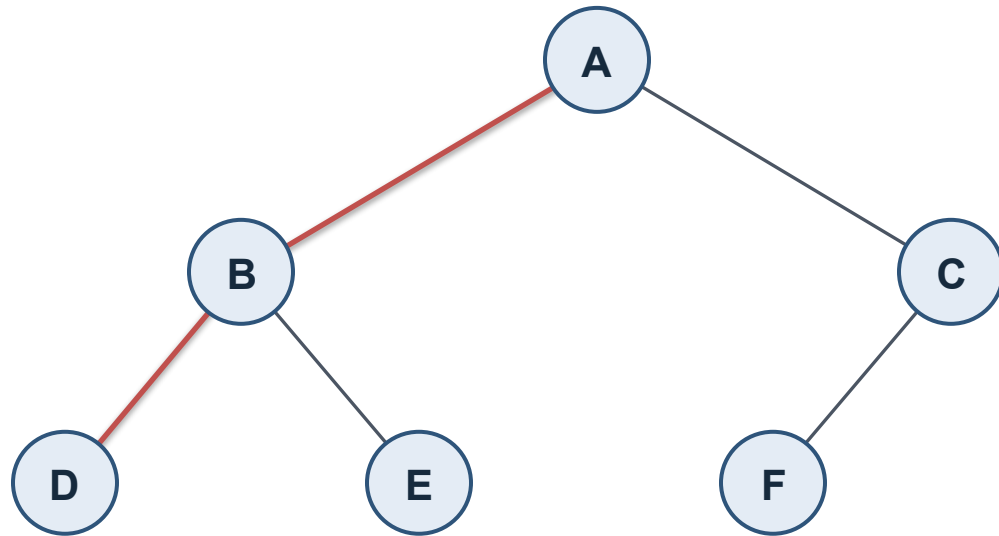
- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B
- Siblings — nodes sharing the same parent (D and E)

Tree Terminology — Structure



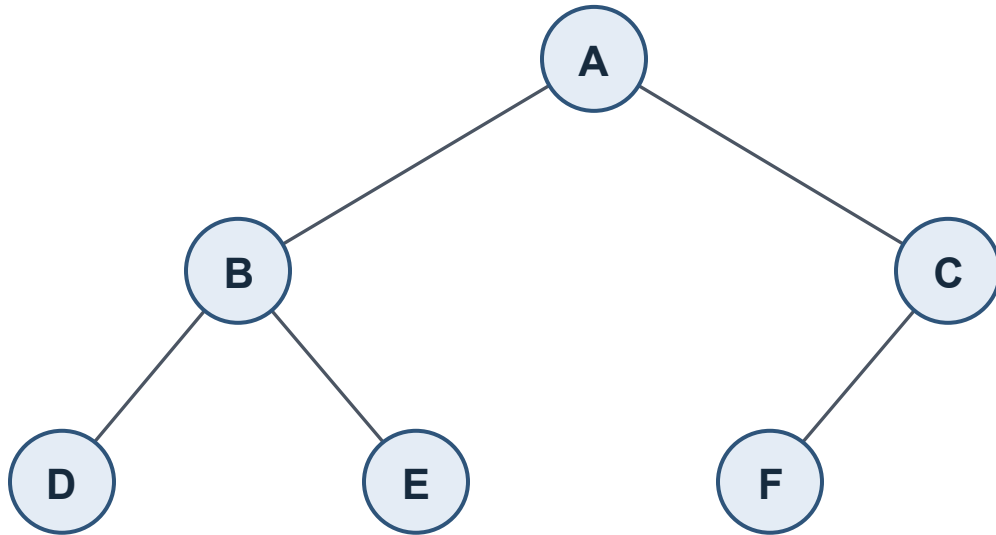
- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B
- Siblings — nodes sharing the same parent (D and E)
- Edge — the connection between a parent and a child

Tree Terminology — Structure



- Root — the node with no parent (A)
- Parent / child — B is the parent of D and E; D and E are children of B
- Siblings — nodes sharing the same parent (D and E)
- Edge — the connection between a parent and a child
- Path — a sequence of nodes connected by edges, e.g. $A \rightarrow B \rightarrow D$

Tree Terminology — Measuring the Tree

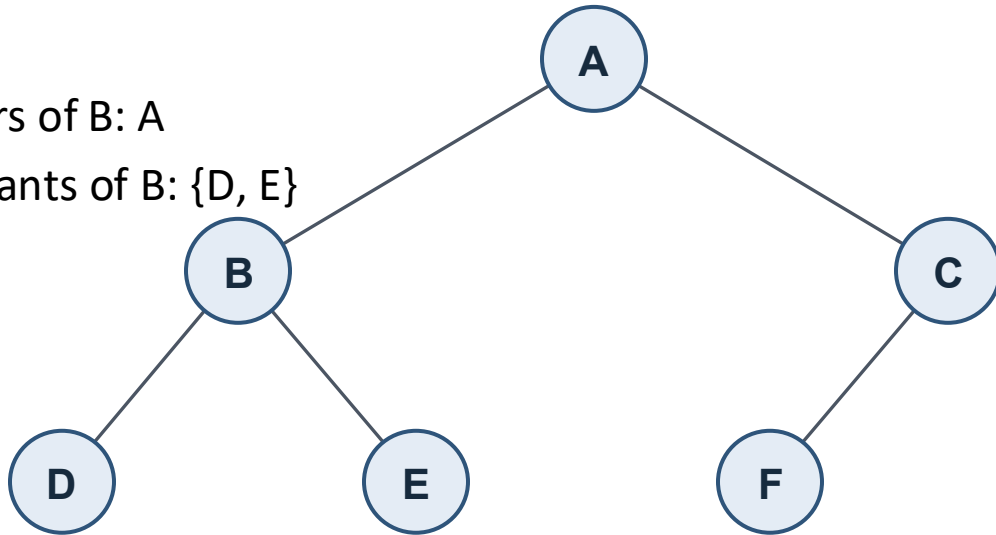


- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)

Tree Terminology — Measuring the Tree

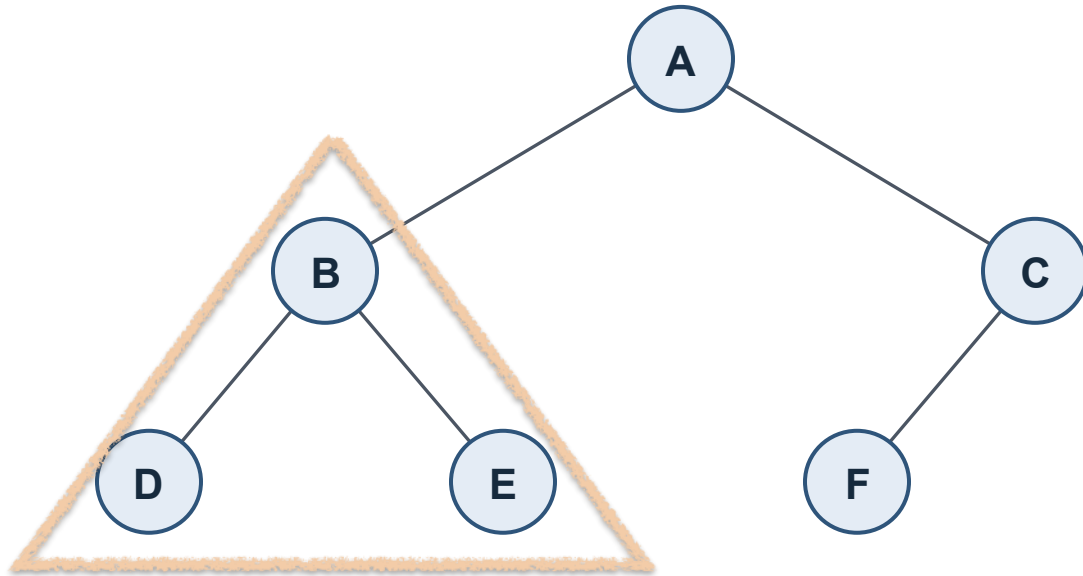
Ancestors of B: A

Descendants of B: {D, E}



- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)
- Ancestors of a node — its parent, grandparent, ... up to the root
- Descendants of a node — its children, grandchildren, ...

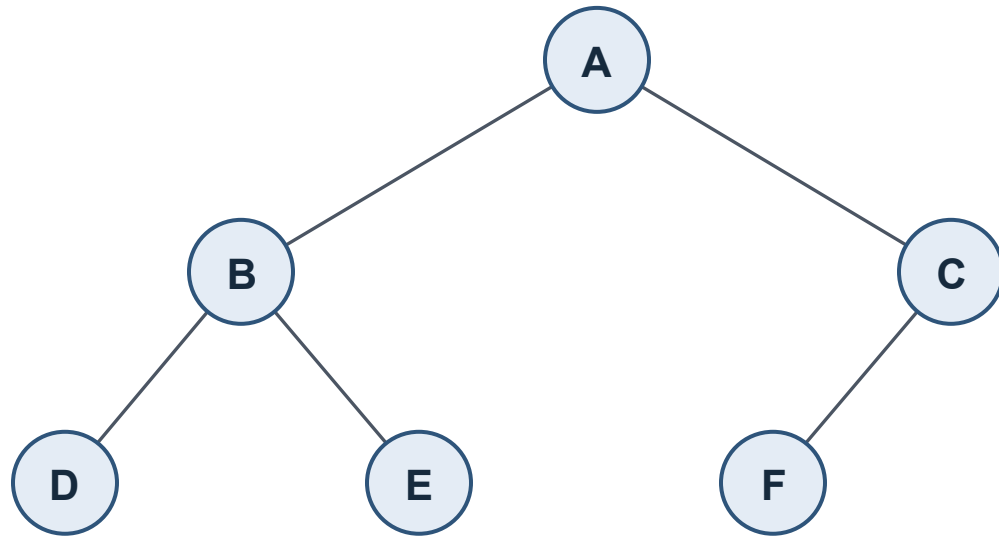
Tree Terminology — Measuring the Tree



- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)
- Ancestors of a node — its parent, grandparent, ... up to the root
- Descendants of a node — its children, grandchildren, ...
- Subtree — a node together with all of its descendants (the highlighted subtree rooted at B)



Tree Terminology — Measuring the Tree



Depth of B = 1

Depth of A = 0

Height of the tree = 2

- Internal node — has at least one child (A, B, C)
- External node / leaf — has no children (D, E, F)
- Ancestors of a node — its parent, grandparent, ... up to the root
- Descendants of a node — its children, grandchildren, ...
- Subtree — a node together with all of its descendants (the highlighted subtree rooted at B)
- Depth of a node — number of ancestors
- Height of the tree — max depth of any node



Representing a Tree in Java

TreeNode<T>.java

```
public class TreeNode<T> {  
  
    // Element stored at this node  
    private T element;  
  
    // Reference to the parent  
    // (null if this node is the root)  
    private TreeNode<T> parent;  
  
    // This node's children, in order  
    private List<TreeNode<T>> children;  
}
```

- Each node is an object holding three things: its element, a reference to its parent, and a list of references to its children.
- The parent reference is what makes upward navigation (e.g. “what is my depth?”) possible without re-walking the tree from the root.
- A general tree node needs a list because the number of children is unbounded — contrast this with the binary tree node we'll see shortly, which has exactly two named child references.



The Tree ADT

Generic · Accessor

```
int size()
boolean isEmpty()
Iterator iterator()
Iterable nodes()

Node root()
Node parent(p)
Iterable children(p)
```

Query · Update

```
boolean isInternal(p)
boolean isExternal(p)
boolean isRoot(p)

element replace(p, o)
// data structures that implement
// Tree may add more update methods
```

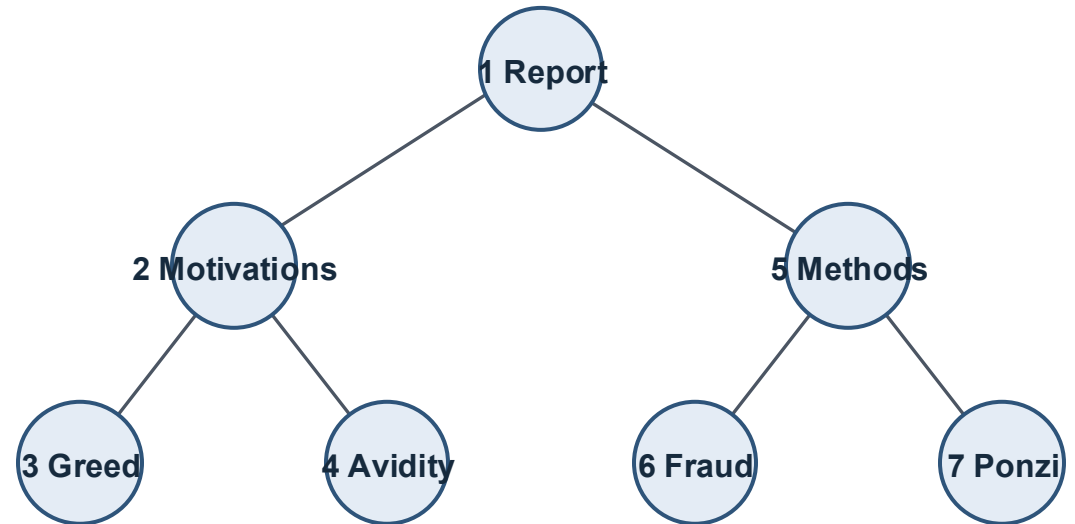
- An Iterator is a cursor: hasNext() checks if there's more, next() advances and returns the next element.
- Iterable just means “can produce an iterator” — it's what makes a for-each loop over the tree possible.

Preorder Traversal

- Visit a node before visiting any of its descendants.
- Application: printing a structured document, where a section must appear before its subsections.

```
Algorithm preOrder(v)
  visit(v)
  for each child w of v
    preOrder(w)
```

Visit order:

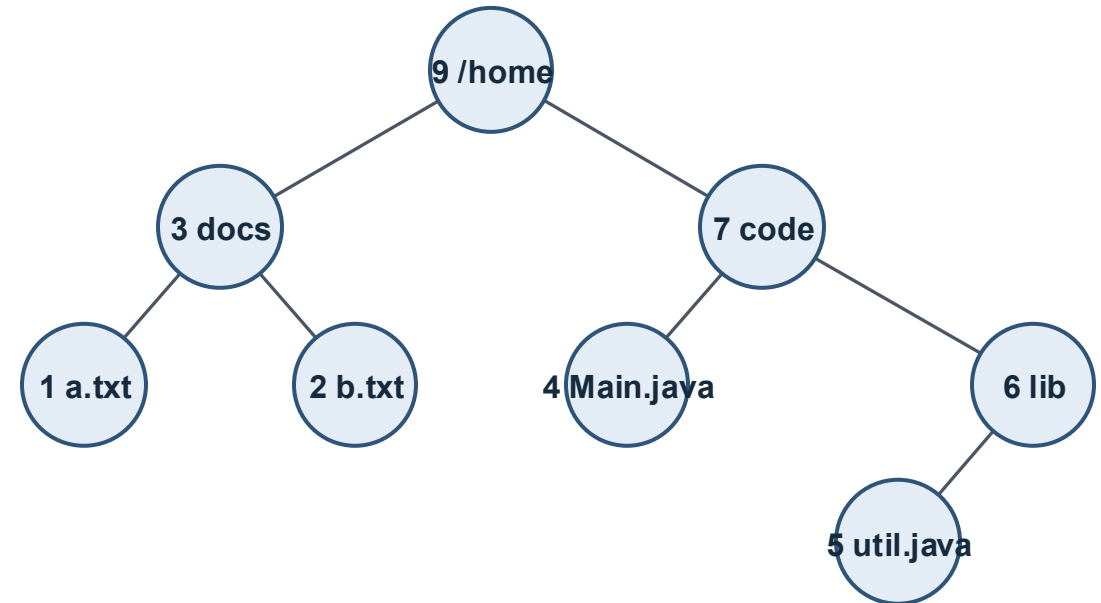


Postorder Traversal

- Visit a node after visiting all of its descendants.
- Application: computing the total disk space used by a directory — you need every subdirectory's total before you can report the parent's.

```
Algorithm postOrder(v)
  for each child w of v
    postOrder(w)
  visit(v)
```

Visit order (folder sizes computed bottom-up):





Which Traversal Do I Need?

Preorder*Visit node, then children*

Copying/serializing a tree; printing outline-style structure

Postorder*Visit children, then node*

Deleting a tree safely; computing a value that depends on all descendants (folder size, subtree height)

Inorder — binary trees only *Visit left subtree, node, right subtree*

Reading a binary search tree's keys out in sorted order (Section 03)

All three visit every node exactly once, so all three run in $\Theta(n)$ time regardless of the tree's shape.

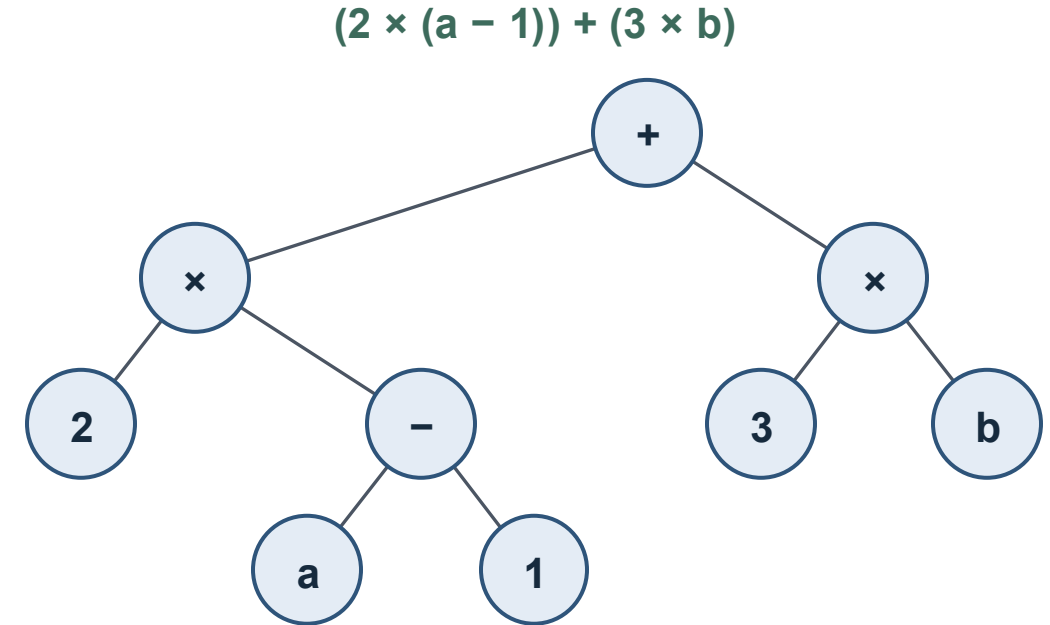
02

Binary Trees

The special case where every node has at most two, ordered children

Binary Trees

- A binary tree is a tree in which every internal node has at most two children.
- The two children are an ordered pair — we call them the left child and the right child, and a node with only one child still distinguishes which side it's on.
- Proper (a.k.a. full) binary tree: every internal node has exactly two children.
- Applications:
 - Arithmetic expression trees (below)
 - Decision processes (yes/no questions)
 - Searching — the entire rest of this module





Properties of Proper Binary Trees

Notation

- n — number of nodes
- e — number of external nodes (leaves)
- i — number of internal nodes
- h — height

Read the last line as the payoff: with n nodes, height can be as low as about $\log_2(n)$ — but nothing here forces it to be that low. A tree can just as easily be a straight chain with $h = n-1$. That gap is exactly what Sections 04–06 close.

Because every internal node has exactly 2 children:

- $e = i + 1$
- $n = 2e - 1$
- $h \leq i$
- $h \leq (n - 1) / 2$
- $e \leq 2^h$
- $h \geq \log_2 e$
- $h \geq \log_2(n + 1) - 1$



The BinaryTree ADT

BinaryTree extends Tree — it inherits every Tree method, plus position-aware accessors:

```
Node left(p)
Node right(p)
boolean hasLeft(p)
boolean hasRight(p)
```

This is the key upgrade over the general Tree ADT: children now have identity (left vs. right), not just membership in an unordered list. Every algorithm for the rest of this module leans on that distinction.

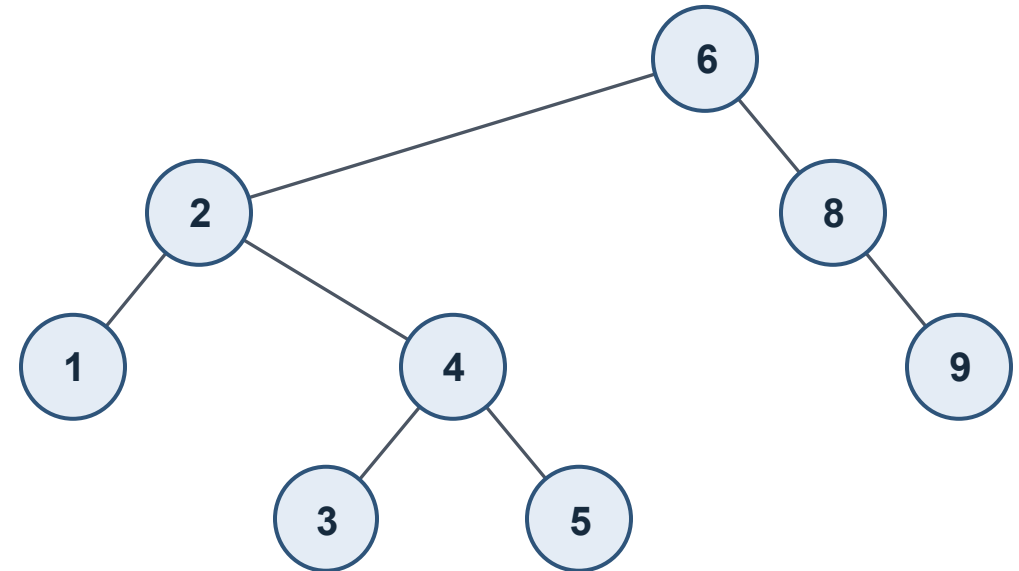


Inorder Traversal

- Visit the left subtree, then the node itself, then the right subtree.

```
Algorithm inOrder(v)
  if hasLeft(v)
    inOrder(left(v))
  visit(v)
  if hasRight(v)
    inOrder(right(v))
```

Visits 1, 2, 3, 4, 5, 6, 8, 9 — in order:





The Linked Representation in Java

BinaryTree.java (excerpt)

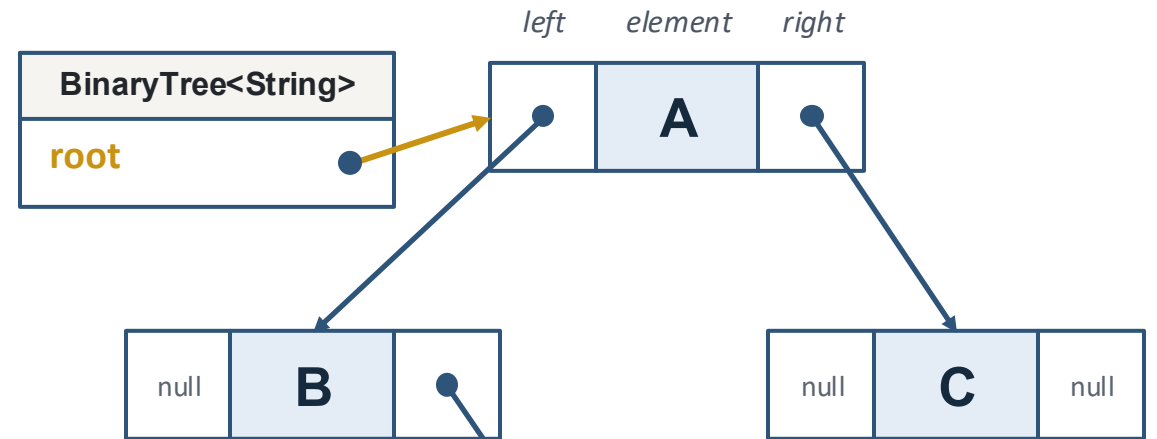
```
public class BinaryTree<T> {  
    public static class Node<T> {  
        private T element;  
        private Node<T> left;  
        private Node<T> right;  
        // no parent reference  
  
        public Node(T element) {  
            this.element = element;  
        } // left, right start null  
    }  
  
    private Node<T> root;  
}
```



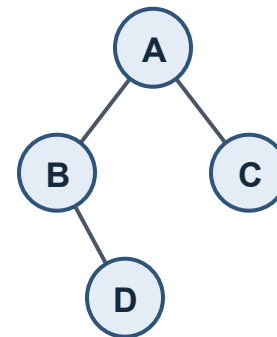
The Linked Representation in Java

BinaryTree.java (excerpt)

```
public class BinaryTree<T> {  
    public static class Node<T> {  
        private T element;  
        private Node<T> left;  
        private Node<T> right;  
        // no parent reference  
  
        public Node(T element) {  
            this.element = element;  
        } // left, right start null  
    }  
  
    private Node<T> root;  
}
```



Drawn as a tree:



Each box is one Node object. Each arrow is a reference stored in a left or right field; null means “no child.”



The Linked Representation in Java

BinaryTree.java (excerpt)

```
public class BinaryTree<T> {  
    public static class Node<T> {  
        private T element;  
        private Node<T> left;  
        private Node<T> right;  
        // no parent reference  
  
        public Node(T element) {  
            this.element = element;  
        } // left, right start null  
    }  
  
    private Node<T> root;  
}
```

Connecting the Binary Tree ADT to this data structure

ADT **root()**

Java **tree.getRoot()**

ADT **left(p), right(p)**

Java **node.getLeft(), node.getRight()**

ADT **hasLeft(p)**

Java **node.getLeft() != null**



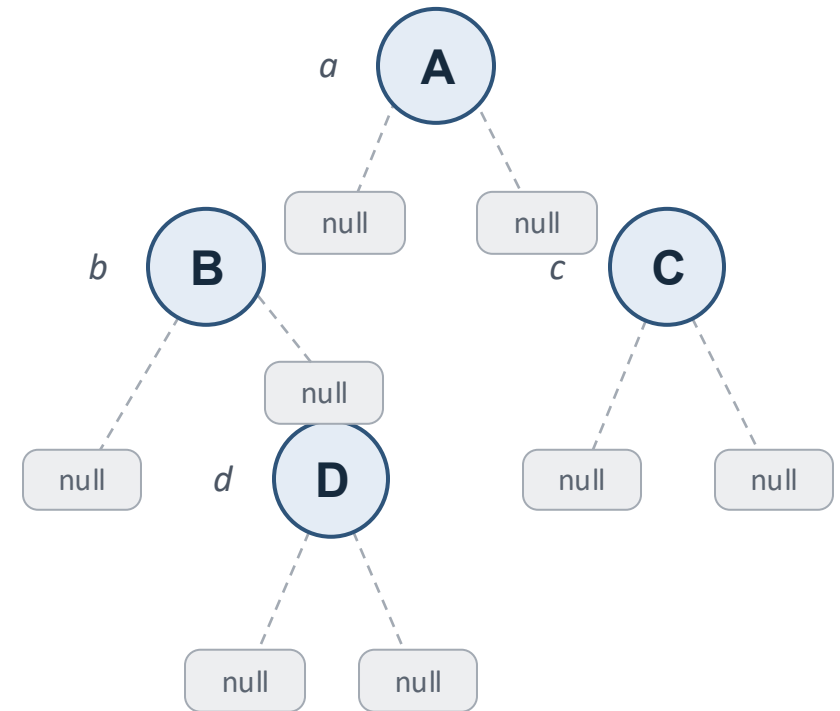
Code Walkthrough: Building a Tree

Build the tree node by node

```
BinaryTree.Node<String> a = new BinaryTree.Node<>("A");  
BinaryTree.Node<String> b = new BinaryTree.Node<>("B");  
BinaryTree.Node<String> c = new BinaryTree.Node<>("C");  
BinaryTree.Node<String> d = new BinaryTree.Node<>("D");
```

- A new Node starts with left == null and right == null.
null means “no child here”: there is no object.

The tree it builds:



 = a null reference (missing child)



Code Walkthrough: Building a Tree

Build the tree node by node

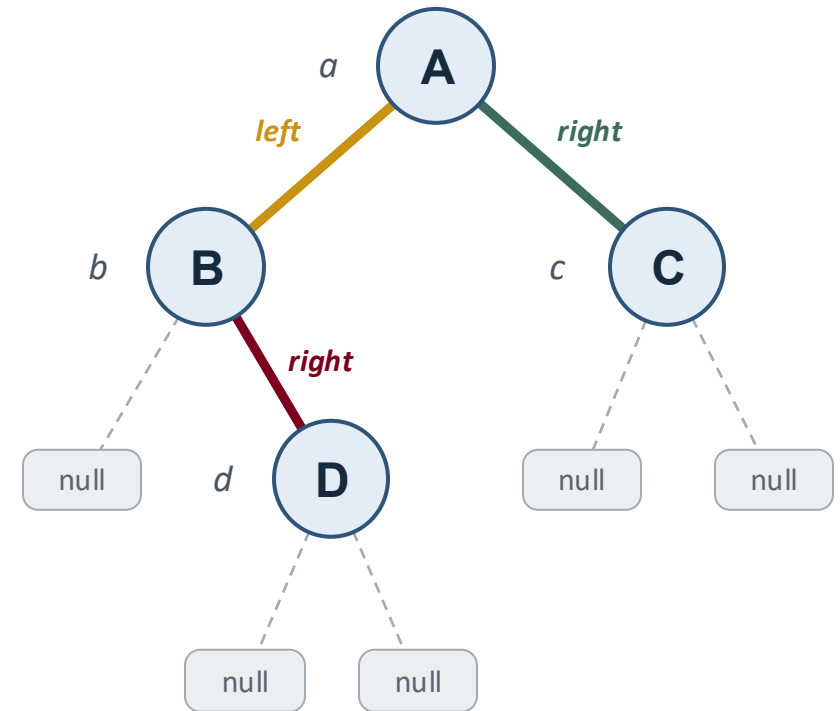
```
BinaryTree.Node<String> a = new BinaryTree.Node<>("A");  
BinaryTree.Node<String> b = new BinaryTree.Node<>("B");  
BinaryTree.Node<String> c = new BinaryTree.Node<>("C");  
BinaryTree.Node<String> d = new BinaryTree.Node<>("D");
```

```
a.setLeft(b);    // edge A-B
```

```
a.setRight(c);   // edge A-C
```

```
b.setRight(d);   // edge B-D; b.left stays null
```

The tree it builds:



null = a null reference (missing child)



Code Walkthrough: Building a Tree

Build the tree node by node

```
BinaryTree.Node<String> a = new BinaryTree.Node<>("A");  
BinaryTree.Node<String> b = new BinaryTree.Node<>("B");  
BinaryTree.Node<String> c = new BinaryTree.Node<>("C");  
BinaryTree.Node<String> d = new BinaryTree.Node<>("D");
```

```
a.setLeft(b);    // edge A-B
```

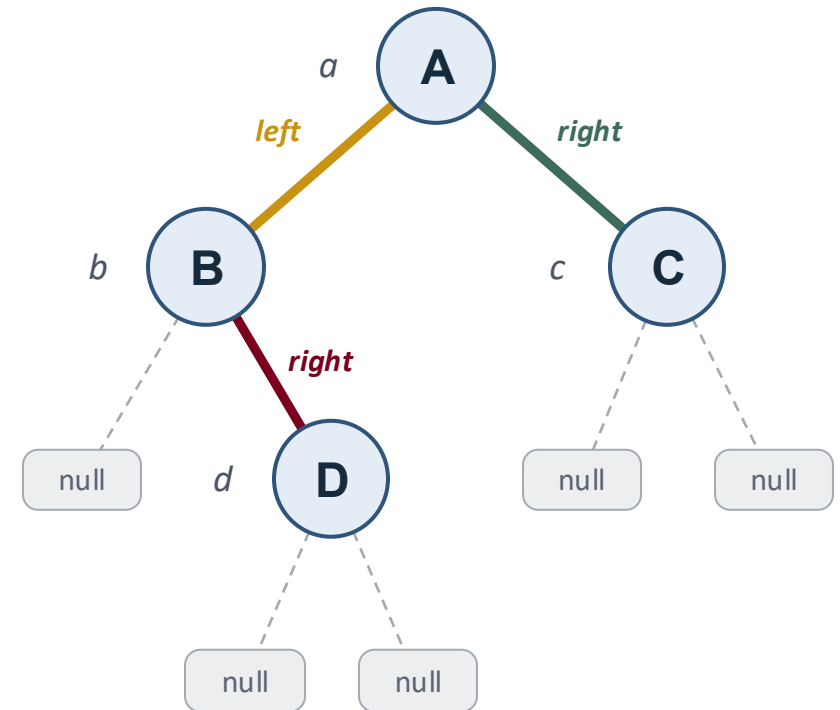
```
a.setRight(c);   // edge A-C
```

```
b.setRight(d);   // edge B-D; b.left stays null
```

```
BinaryTree<String> tree = new BinaryTree<>(a);  
tree.printTree();           // next slide
```

- The tree object stores only root; every other node is reached by following references.

The tree it builds:



null = a null reference (missing child)



Inspecting the Structure with printTree()

Console output of `tree.printTree()`

```
A
├── L: B
│   └── R: D
└── R: C
```

- `printTree()` is provided in `BinaryTree.java` so you can check the links you set. You are not expected to write its branch formatting (`├──` `└──` `|`).
- L: and R: show which field holds each child. B has no L: line because `b.getLeft()` is null.
- **Quick check:** Which traversal pattern does this function follow? (see code)

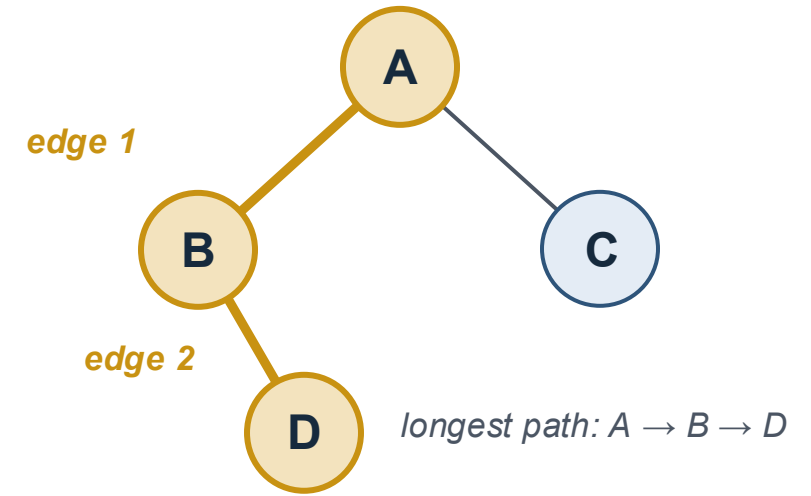


Height in Java: height()

BinaryTree.java

```
// Height is measured in edges: a leaf has height 0.  
// An empty tree has height -1.  
public int height() {  
    return height(root);  
}  
  
private int height(Node<T> node) {  
    if (node == null) return -1;  
    int leftHeight = height(node.getLeft());  
    int rightHeight = height(node.getRight());  
    return 1 + Math.max(leftHeight, rightHeight);  
}
```

Example: height() returns 2



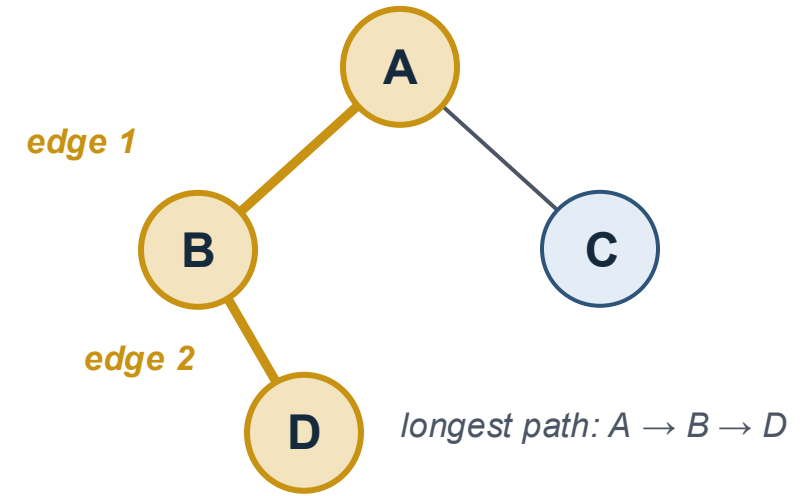


Height in Java: height()

BinaryTree.java

```
// Height is measured in edges: a leaf has height 0.  
// An empty tree has height -1.  
public int height() {  
    return height(root);  
}  
  
private int height(Node<T> node) {  
    if (node == null) return -1;  
    int leftHeight = height(node.getLeft());  
    int rightHeight = height(node.getRight());  
    return 1 + Math.max(leftHeight, rightHeight);  
}
```

Example: height() returns 2



Convention used in the code

empty tree $\rightarrow -1$ leaf $\rightarrow 0$

leaf: $1 + \max(-1, -1) = 0$



Height in Java: height()

BinaryTree.java

```
// Height is measured in edges: a leaf has height 0.
```

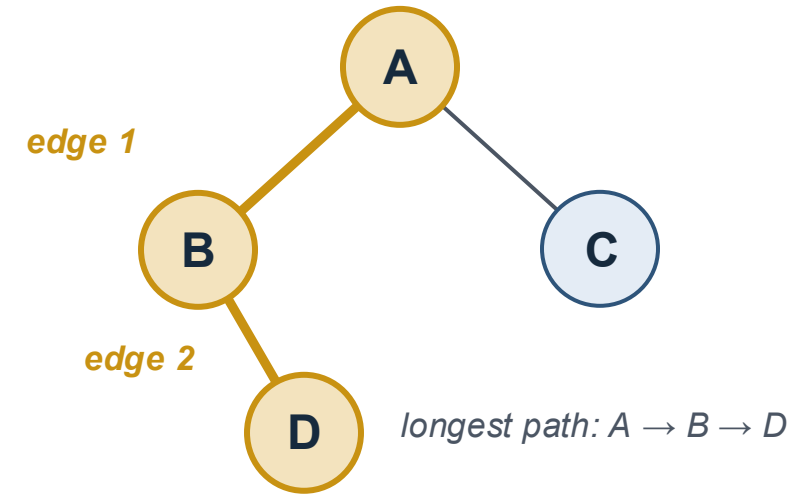
```
// An empty tree has height -1.
```

```
public int height() {  
    return height(root);  
}
```

```
private int height(Node<T> node) {  
    if (node == null) return -1;  
    int leftHeight = height(node.getLeft());  
    int rightHeight = height(node.getRight());  
    return 1 + Math.max(leftHeight, rightHeight);  
}
```

- **Base case:** an empty subtree (null) has height -1.
- **Recursive case:** get both children's heights, then add 1 for the edge down to the taller one.

Example: height() returns 2



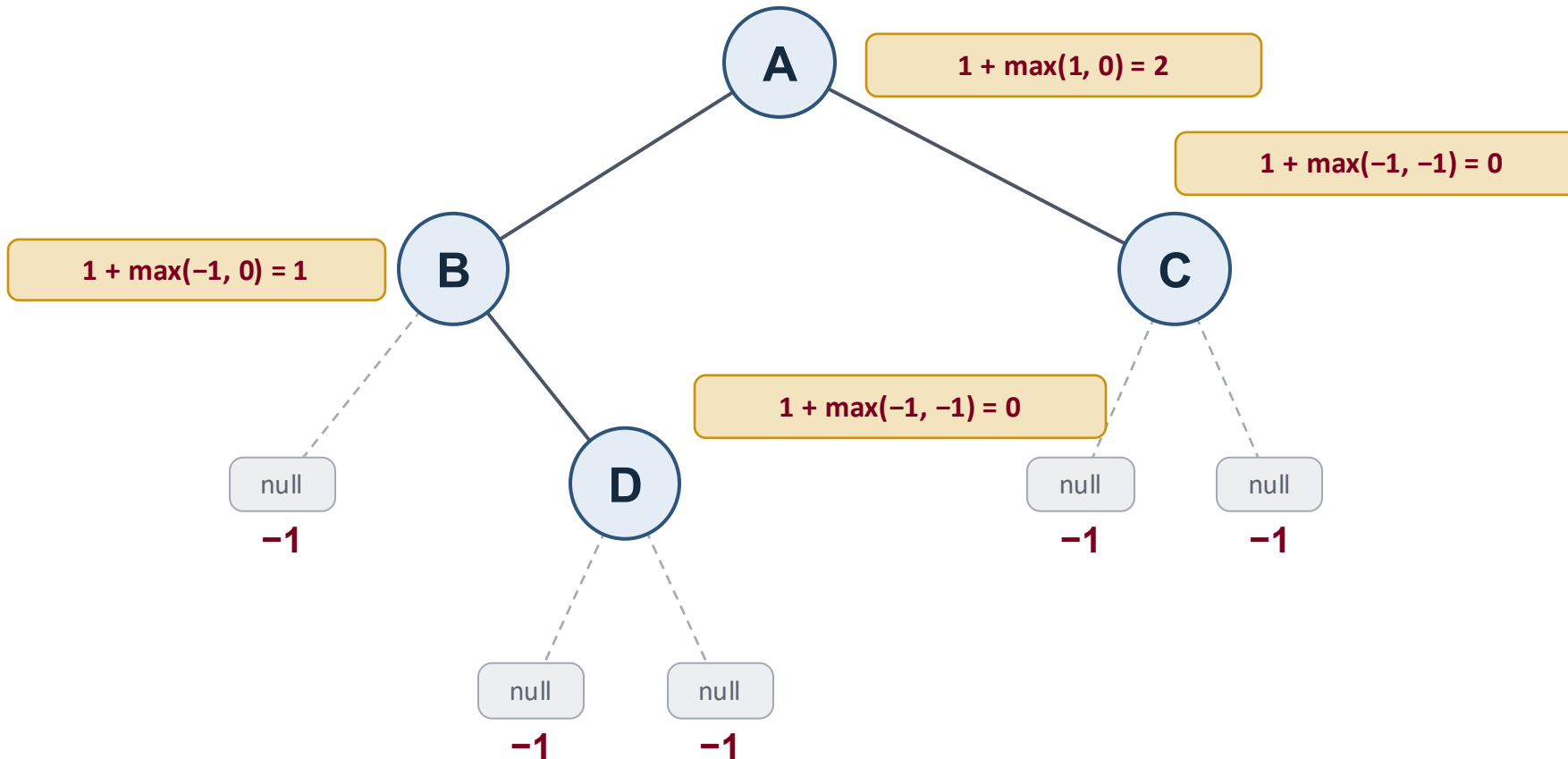
Convention used in the code

empty tree $\rightarrow -1$ leaf $\rightarrow 0$

leaf: $1 + \max(-1, -1) = 0$



Tracing height(): Return Values Flow Up



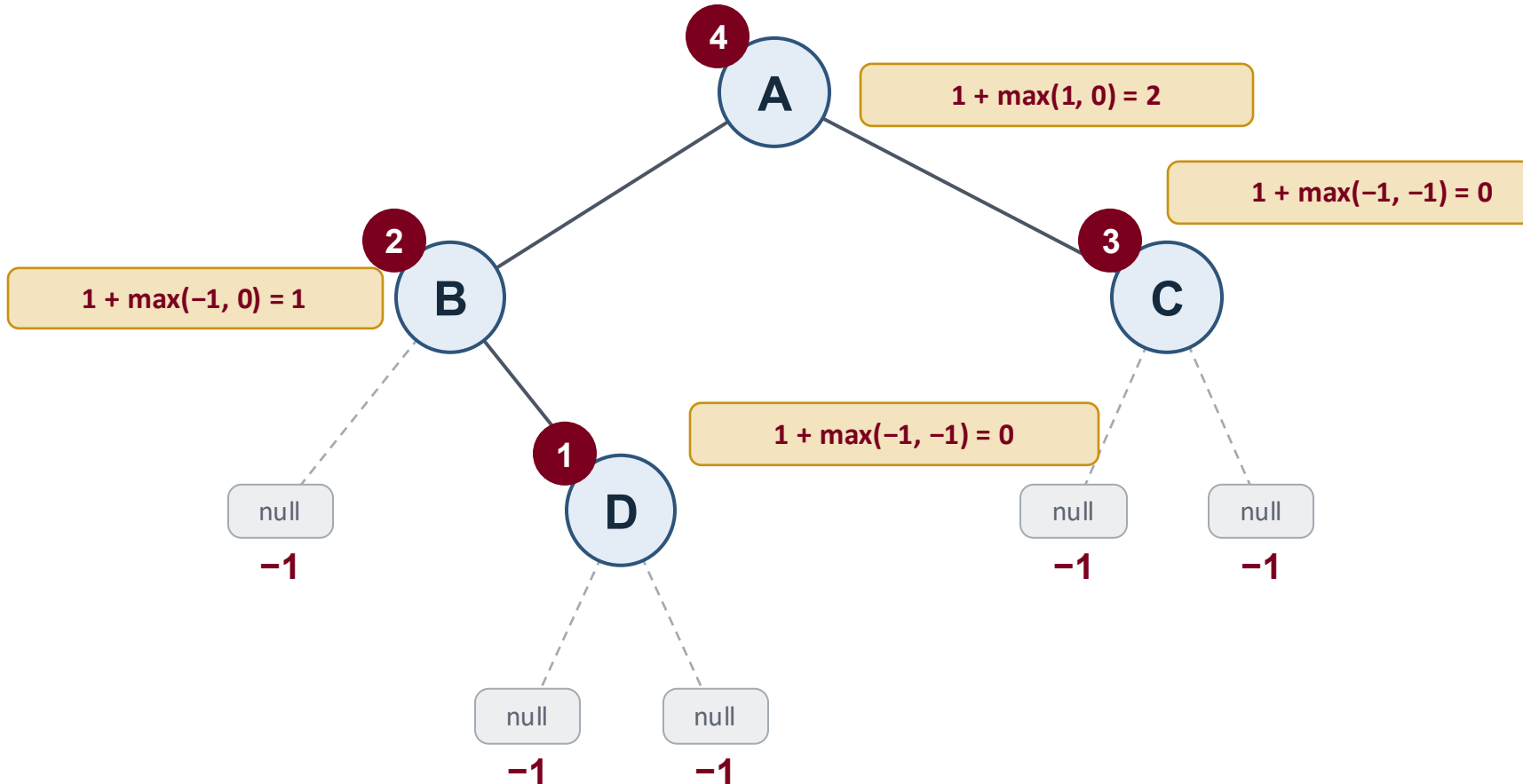
Question:

Which traversal pattern does height() follow: preorder, inorder, or postorder?

Values flow up: each call returns one int to its caller, from the nulls to the root.



Reveal: height() Is a Postorder Computation



Postorder

A node's result is computed only **after** both child results come back: left, right, then the node.

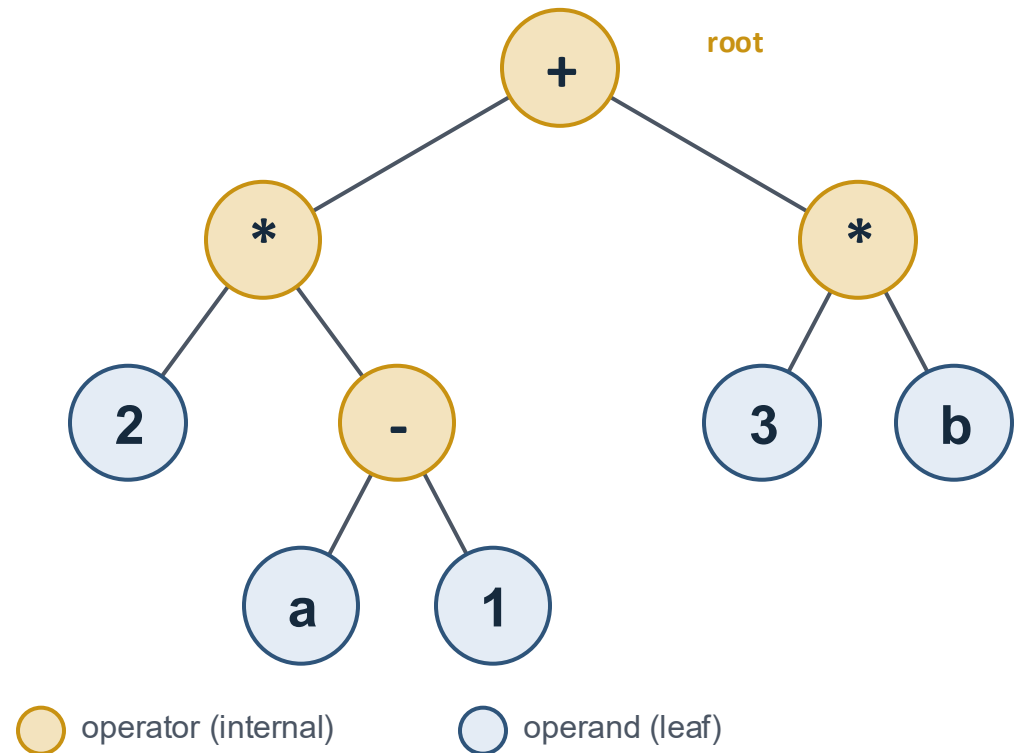
Values flow up: each call returns one int to its caller, from the nulls to the root.



Arithmetic Expression Trees

Binary Tree representing Arithmetic expression:

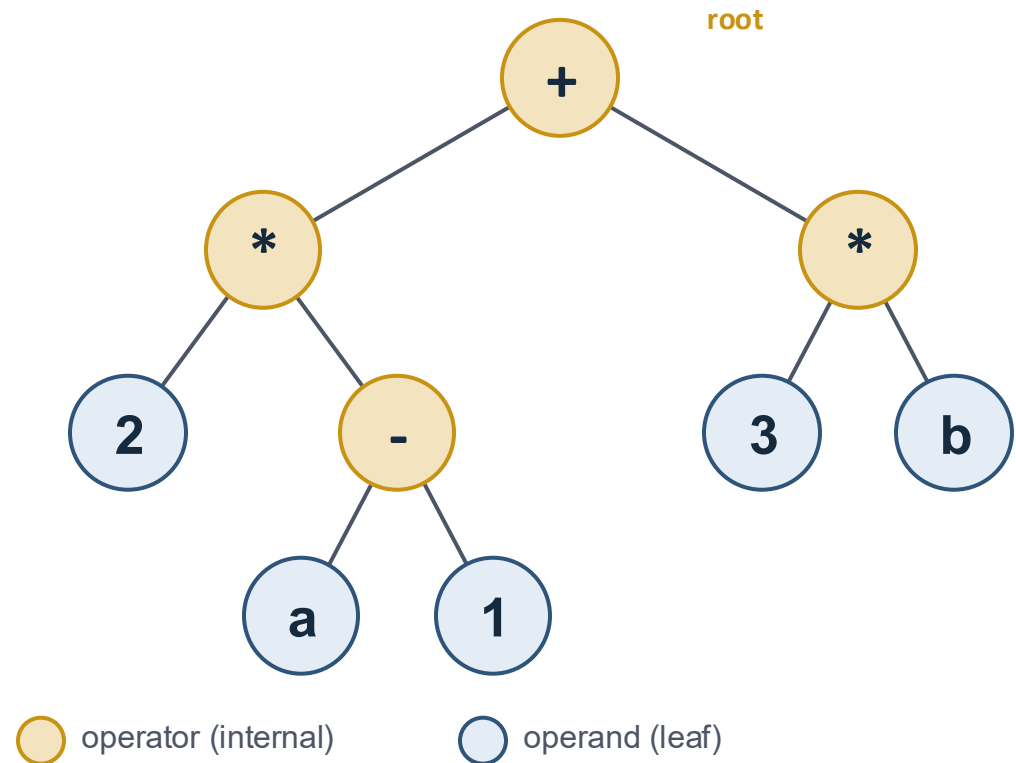
- Leaves hold operands
- internal nodes hold operators





Arithmetic Expression Trees

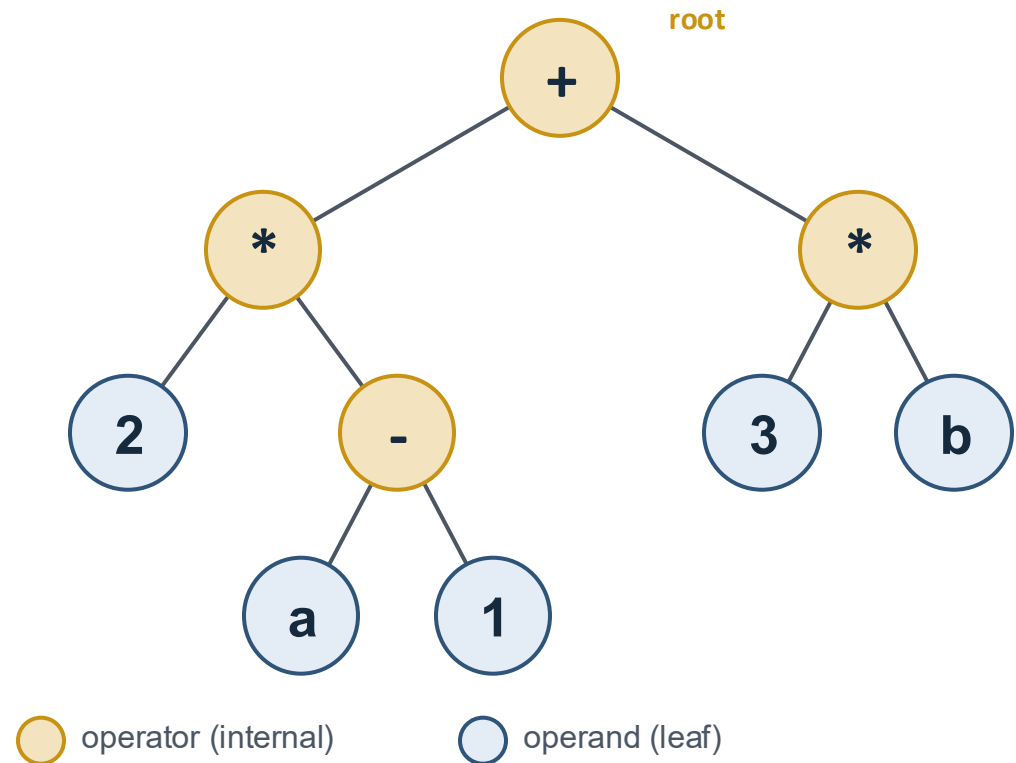
- Read this tree inorder and the tokens appear in the order we write them: $2 * a - 1 + 3 * b$





Arithmetic Expression Trees

- Read this tree inorder and the tokens appear in the order we write them: **2 * a - 1 + 3 * b**
- But that loses the structure: Parentheses put it back where each operator node wraps its own subexpression.



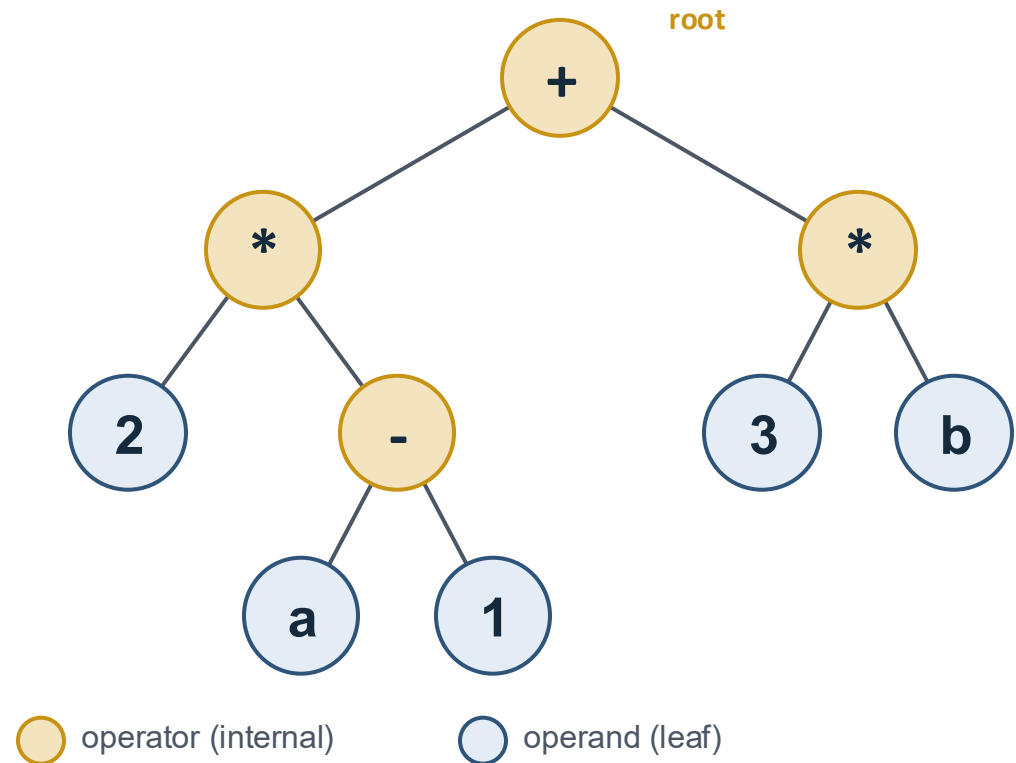


Arithmetic Expression Trees

- Read this tree inorder and the tokens appear in the order we write them: $2 * a - 1 + 3 * b$
- But that loses the structure: Parentheses put it back where each operator node wraps its own subexpression.

Expected result:

$((2 * (a - 1)) + (3 * b))$





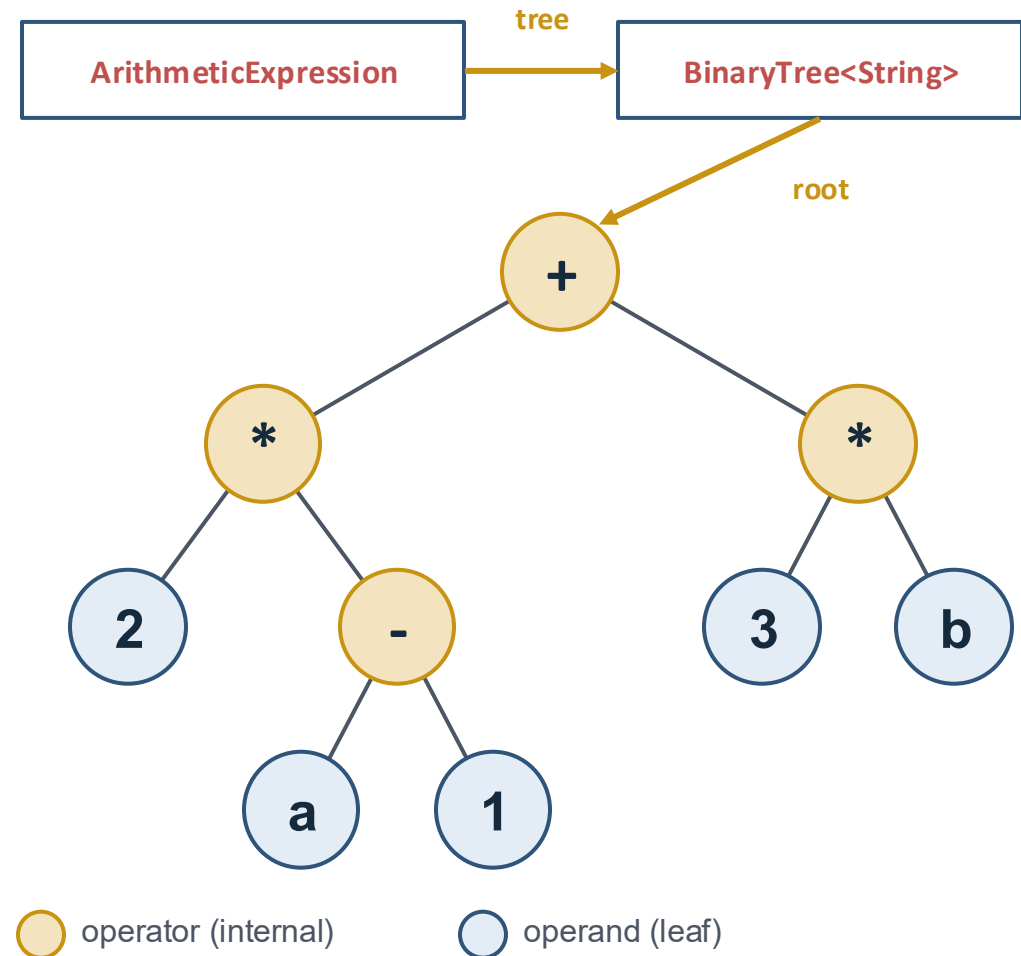
Arithmetic Expression Trees

- Read this tree inorder and the tokens appear in the order we write them: $2 * a - 1 + 3 * b$
- But that loses the structure: Parentheses put it back where each operator node wraps its own subexpression.

Expected result:

$((2 * (a - 1)) + (3 * b))$

- `ArithmeticExpression` Class has a `BinaryTree<String>` field named `tree`.





Activity: toExpressionString()

≈ 12 min

ArithmeticExpression.java — complete the helper

```
public String toExpressionString() {  
    return toExpressionString(tree.getRoot());  
}  
  
private String toExpressionString(  
    BinaryTreeNode<String> node) {  
    // TODO: your code here  
    return ""; // Replace this line.  
}
```

Hints

Leaf (no children) = an operand → return its element.

Otherwise: build the left and right expressions recursively, then combine them with the operator, inside ().

- 1 Complete the private recursive helper.
- 2 **Return** a String. Don't print pieces during the recursion; build the answer and return it.
- 3 Run main and compare with the expected string:
$$((2 * (a - 1)) + (3 * b))$$

Done when main prints **Matches expected: true**
- 4 **Finished early?** What should toExpressionString() return for an empty tree?

03

Binary Search Trees

Ordering the tree so that search, insertion, and deletion all cost $O(h)$



Warm-Up: Binary Search

- On a sorted array, binary search finds a key the way you'd play a high–low guessing game: check the middle, then discard half the remaining candidates.



Warm-Up: Binary Search

- On a sorted array, binary search finds a key the way you'd play a high–low guessing game: check the middle, then discard half the remaining candidates.
- In an array, each step really does halve the range, so it takes $O(\log n)$ steps. Example: `find(7)`, with $m = \lfloor (l + h) / 2 \rfloor$.





Warm-Up: Binary Search

- On a sorted array, binary search finds a key the way you'd play a high–low guessing game: check the middle, then discard half the remaining candidates.
- In an array, each step really does halve the range, so it takes $O(\log n)$ steps. Example: find(7), with $m = \lfloor (l + h) / 2 \rfloor$.





Warm-Up: Binary Search

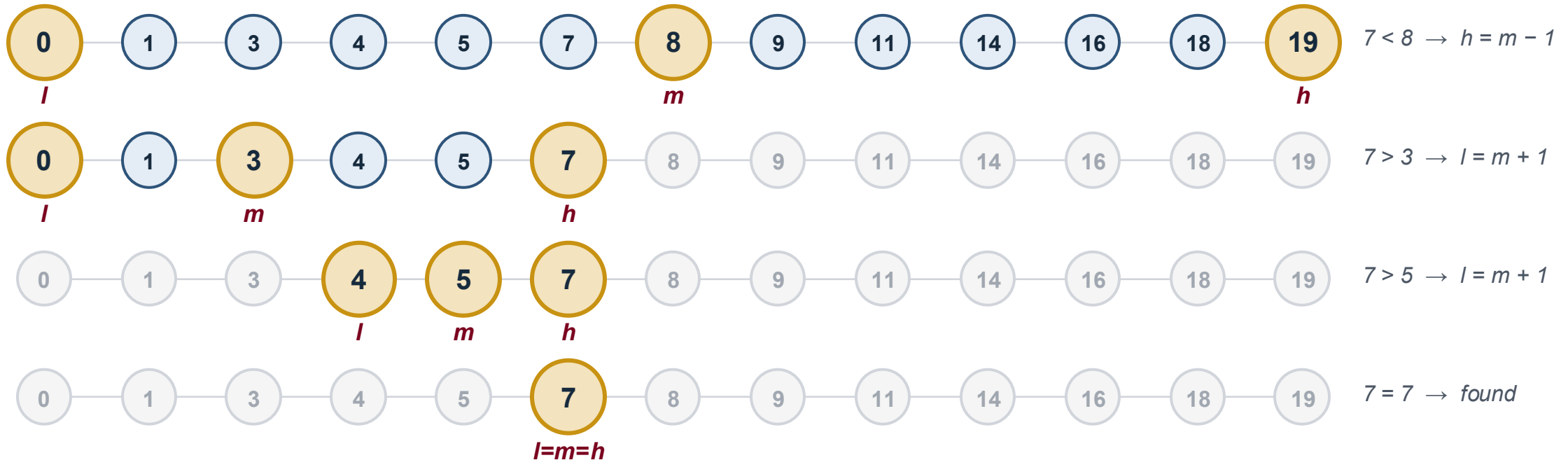
- On a sorted array, binary search finds a key the way you'd play a high–low guessing game: check the middle, then discard half the remaining candidates.
- In an array, each step really does halve the range, so it takes $O(\log n)$ steps. Example: `find(7)`, with $m = \lfloor (l + h) / 2 \rfloor$.





Warm-Up: Binary Search

- On a sorted array, binary search finds a key the way you'd play a high–low guessing game: check the middle, then discard half the remaining candidates.
- In an array, each step really does halve the range, so it takes $O(\log n)$ steps. Example: `find(7)`, with $m = \lfloor (l + h) / 2 \rfloor$.

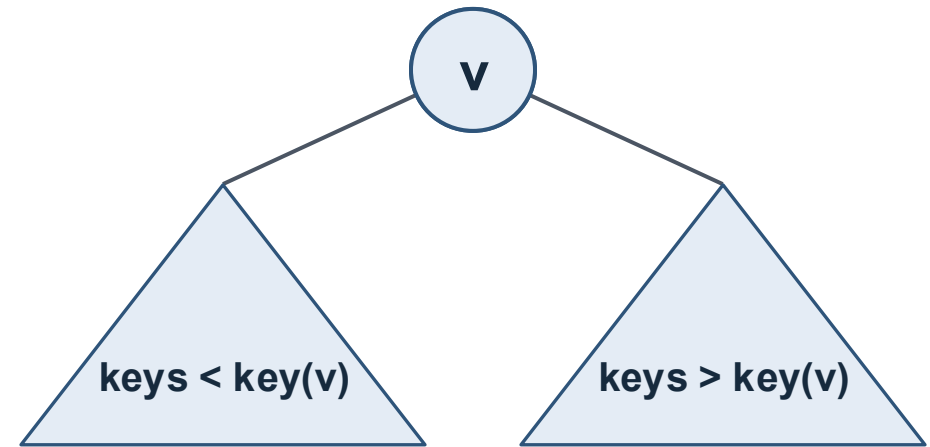


A BST stores these decisions in links: each comparison picks one subtree. Whether that discards about half the keys depends on the tree's shape (slide 30).

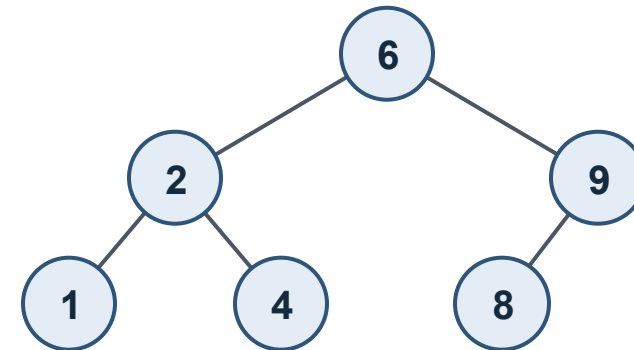


The BST Ordering Property

- A binary search tree (BST) is a binary tree in which each node stores a key, and for every node v :
 - every key in v 's left subtree is $< \text{key}(v)$
 - every key in v 's right subtree is $> \text{key}(v)$
 - (we assume distinct keys)



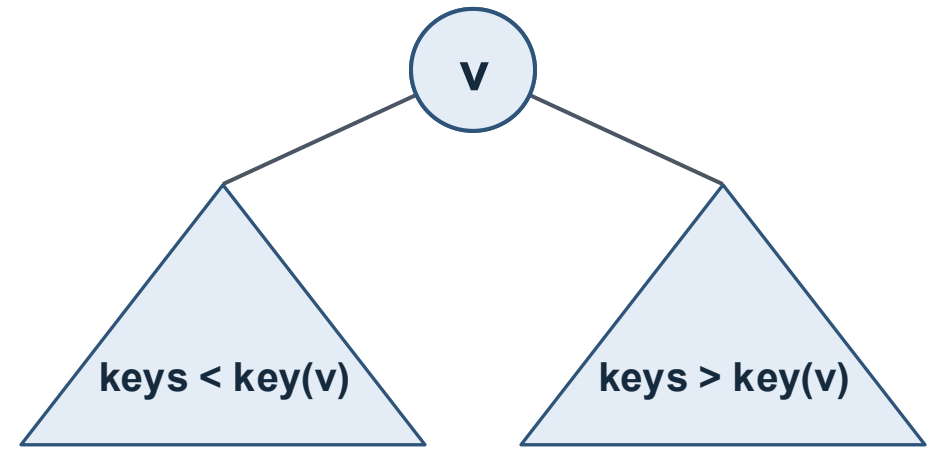
Example (next slide): inorder gives 1, 2, 4, 6, 8, 9



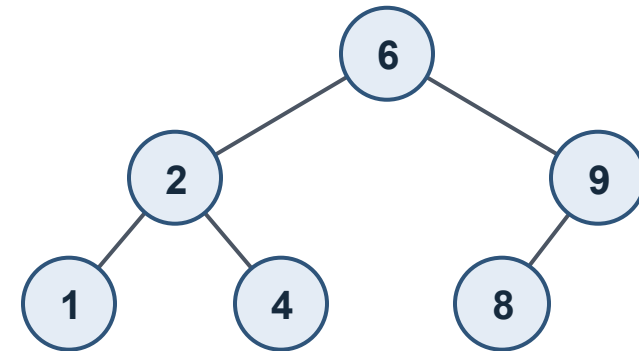


The BST Ordering Property

- A binary search tree (BST) is a binary tree in which each node stores a key, and for every node v :
 - every key in v 's left subtree is $< \text{key}(v)$
 - every key in v 's right subtree is $> \text{key}(v)$
 - *(we assume distinct keys)*
- Same linked structure as before: a missing child is just a null reference.



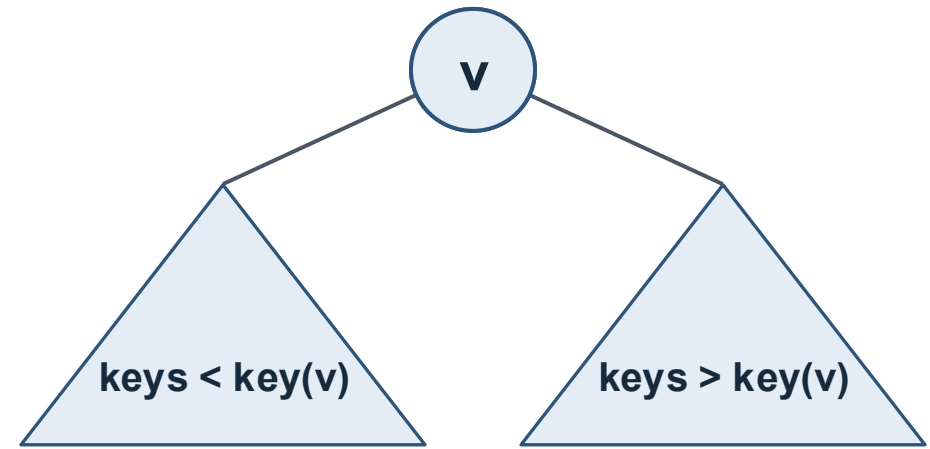
Example (next slide): inorder gives 1, 2, 4, 6, 8, 9



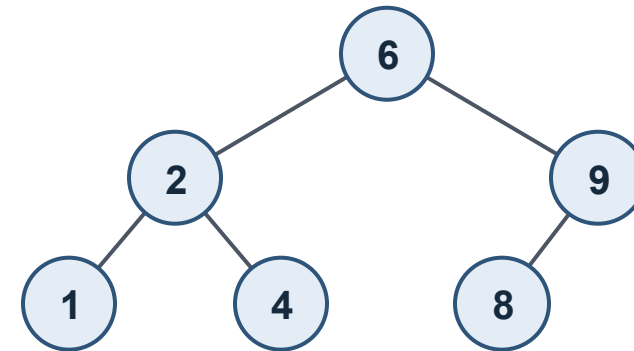


The BST Ordering Property

- A binary search tree (BST) is a binary tree in which each node stores a key, and for every node v :
 - every key in v 's left subtree is $< \text{key}(v)$
 - every key in v 's right subtree is $> \text{key}(v)$
 - (we assume distinct keys)
- Same linked structure as before: a missing child is just a null reference.
- Consequence: an inorder traversal visits the keys in increasing order.



Example (next slide): inorder gives 1, 2, 4, 6, 8, 9





Search in BST: Algorithm and One Trace

First call: `treeSearch(k, root)`

Algorithm `search(k, v)`

if `v == null`

return null // not found

if `k == key(v)`

return v // found

else if `k < key(v)`

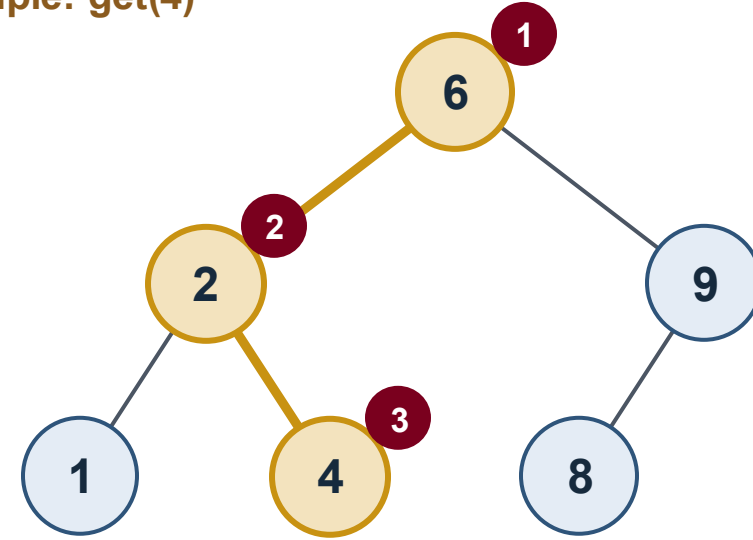
return `search(k, left(v))`

else

return `search(k, right(v))`

- Each comparison sends the search into one subtree. The other subtree is never examined.
- `get(5)`: $6 \rightarrow 2 \rightarrow 4 \rightarrow$ right reference is null $\rightarrow$ not found. That null is where 5 would be inserted.

Example: `get(4)`



1 at 6: $4 < 6 \rightarrow$ go left

2 at 2: $4 > 2 \rightarrow$ go right

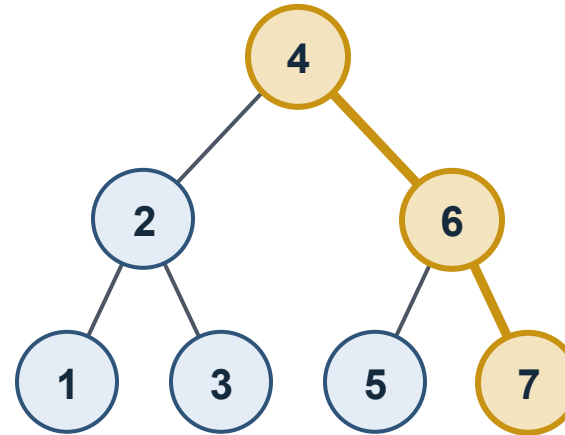
3 at 4: $4 = 4 \rightarrow$ found (3 comparisons)



Search Costs $O(h)$, and h Depends on Shape

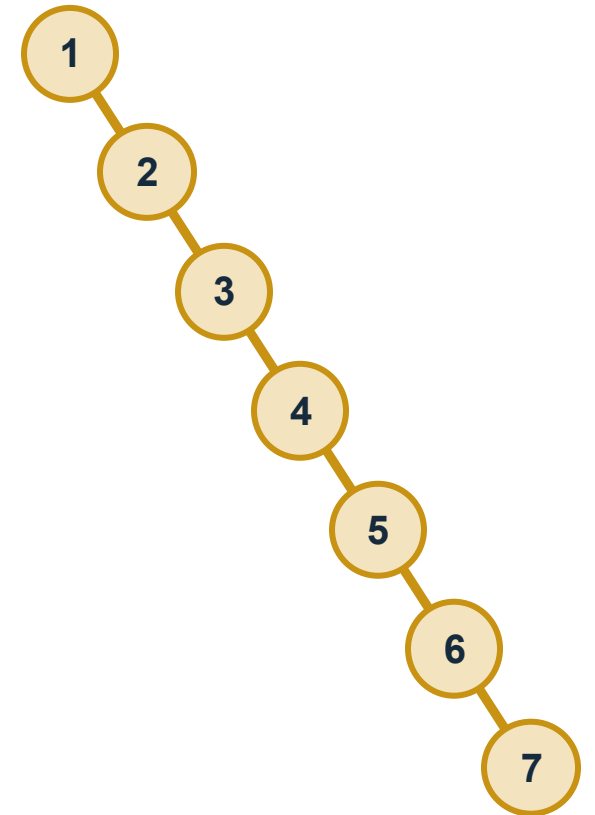
- Search follows one downward path, so it makes at most $h + 1$ comparisons: $O(h)$.
- Each comparison selects one subtree. It does not necessarily discard half the candidates; the skipped subtree may be small or empty.
- Bushy tree: $h \approx \log_2 n$, so search is $O(\log n)$.
- Chain (e.g., keys inserted in sorted order): $h = n - 1$, so search is $O(n)$.

Bushy: $n = 7, h = 2$



find(7): 3 comparisons

Chain: $n = 7, h = 6$



find(7): 7 comparisons

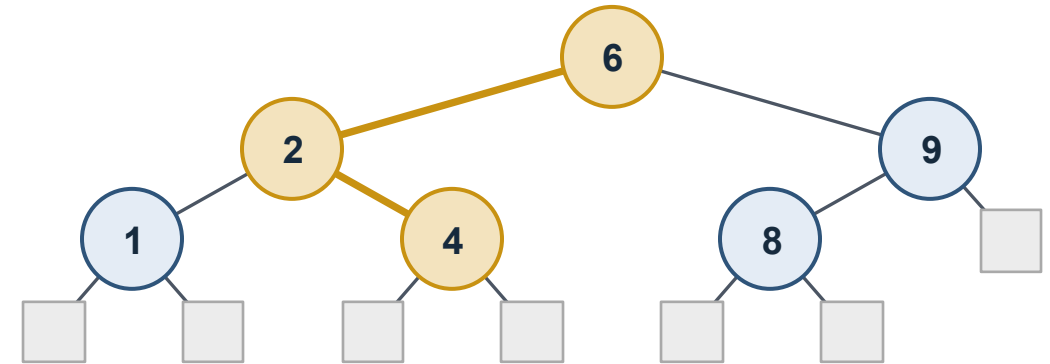
Logarithmic height needs a suitable tree shape. Balanced BSTs such as AVL and Red-Black trees (Trees II) guarantee it.



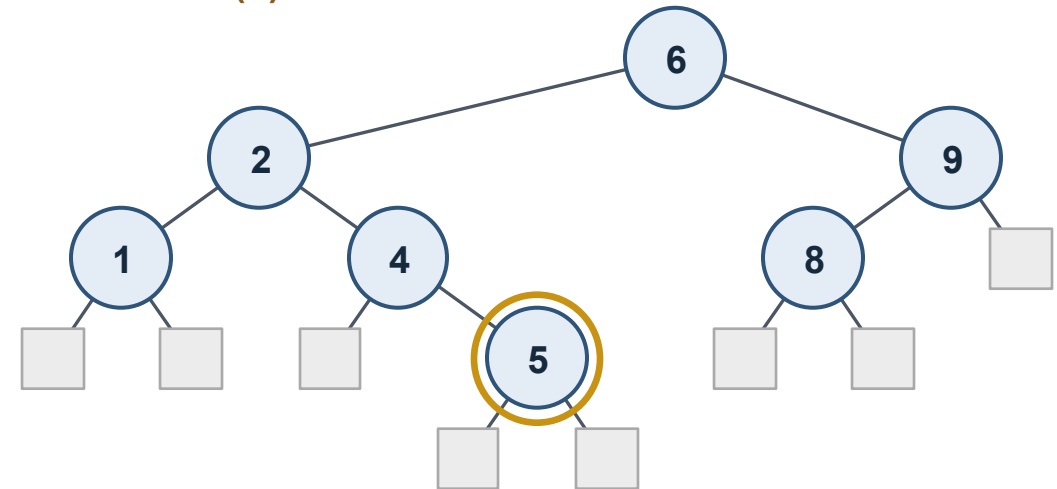
Insertion — Algorithm

- To insert key k ($\text{put}(k, o)$):
 - Run $\text{treeSearch}(k, \text{root})$. If k isn't already present, the search ends at a null reference.
 - Create a new Node for k and store it in that left or right field.
- That's it: the new node is always a leaf, which is why insertion costs $O(h)$, just like search.

Grey squares in the diagrams = null child references.



After insert(5):

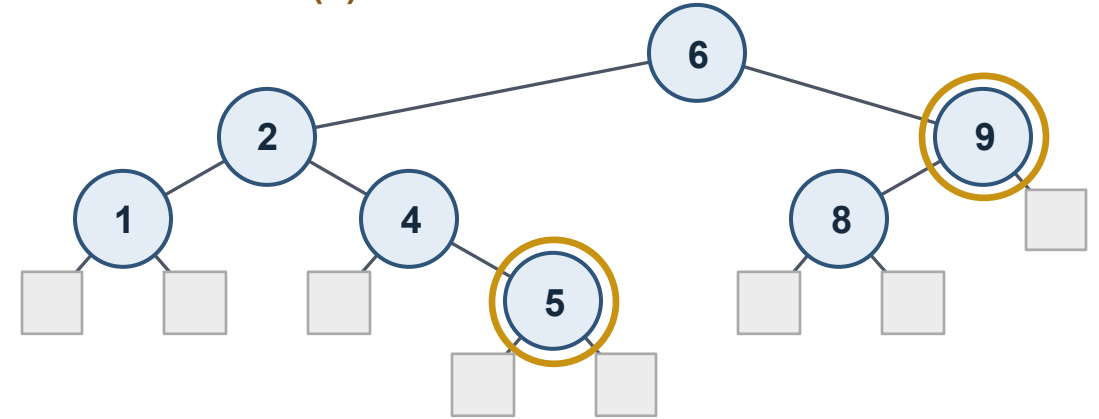




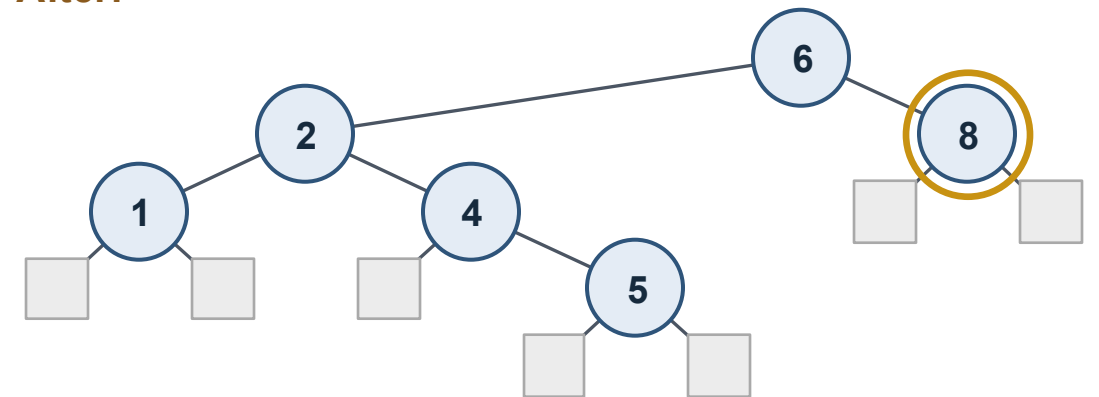
Deletion — Easy Case: At Most One Child

- To remove key k : search for k , landing at the node v that stores it.
- If v has at most one child (the other reference is null), unlink v : v 's parent now points to v 's only child, or to null if v is a leaf.
- Example: `remove(9)`. Node 9's right reference is null, so 9 is removed and its left child, 8, takes its place.

Before: `remove(9)`



After:

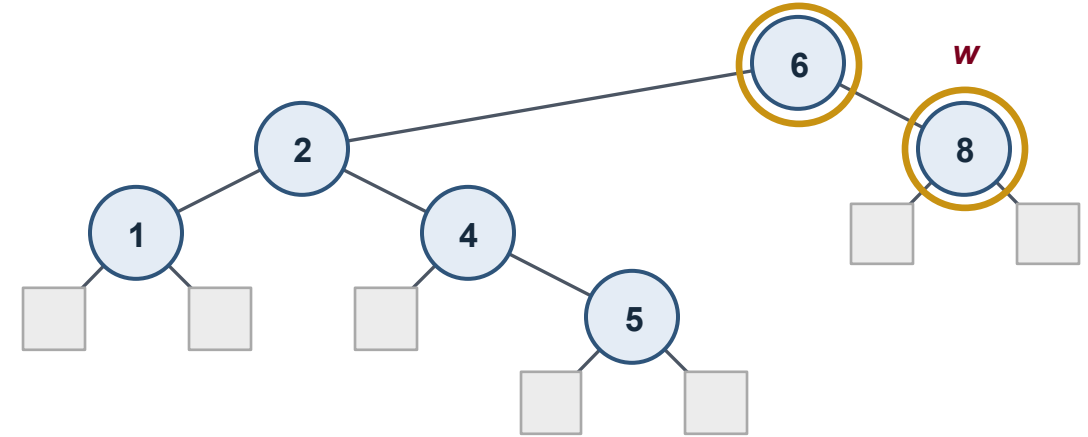




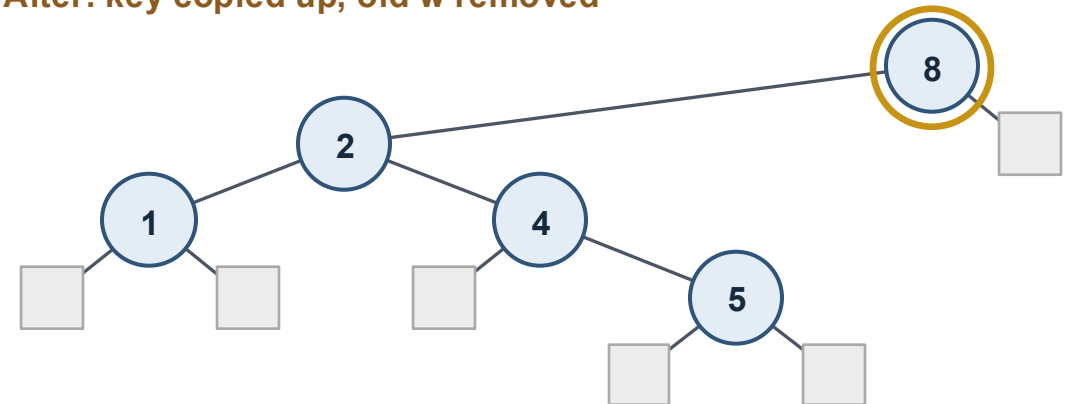
Deletion — Hard Case: Two Children

- If v has two children, we can't just promote one; we need a replacement key that preserves the ordering property.
- Find w , the node immediately after v in an inorder traversal (its inorder successor): go right once, then left as far as possible. w 's left reference is null.
- Copy $\text{key}(w)$ into v , then delete w using the easy case above.
- Example: `remove(6)` from the tree at right. The inorder successor of 6 is 8; copy 8 into the root, then remove the old node 8.

Before: `remove(6)`, $v = \text{root}$, $w = \text{successor}$



After: key copied up, old w removed



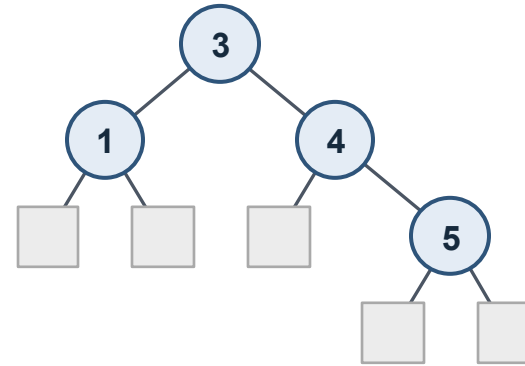


Performance — and a Problem

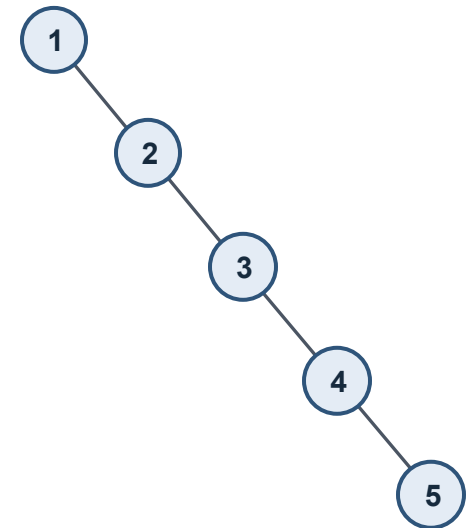
- For a BST with n nodes and height h :
 - space used: $O(n)$
 - get, put, remove: all $O(h)$, since each follows at most one downward path
- The catch: h depends entirely on insertion order.

Insert keys already sorted — 1, 2, 3, 4, 5 — and every node has only a right child. The “tree” degenerates into a linked list: $h = n - 1$, and every operation becomes $O(n)$.

Balanced ($n=5$, $h=2$)



Sorted insert ($n=5$, $h=4$)



This is exactly the gap balanced BSTs close: AVL and Red-Black trees (Trees II) guarantee $h = O(\log n)$ regardless of insertion order.